SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

TABLE OF CONTENTS

1. Introduction: 3
1.1. Purpose: 3
1.2. Background: 3
1.3. Target Stakeholder 4
2. System Architecture & Design 5
2.1. High-Level Architecture 5
3. Functional Requirements & Scope 12
3.1. Minimum Viable Product: 12
3.2. Non-Functional Requirements (NFRs) 12
3.3. Out of Scope / Future Enhancements 14
4. Monitor, Evaluation, Assumptions & Dependencies 14
4.1. Technical Evaluation 14
4.2. Monitoring 15
4.3. Assumptions 16
5. Project Management & Team Contributions 17
5.1. Project Timeline 17
5.2. Team Members Role 18
5.3. Recommendations 18

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This document introduces InvAgent, the strategic technical solution to the fragmented, manual procurement problem faced by Malaysian SMEs, as identified in the Product Requirement Documentation (PRD).

1.1. Purpose:

The System Analysis Document highlights the technical scope and design-related decisions behind InvAgent's development. The key elements covered are:

1. Architecture: A cloud-native, event-driven workflow engine with GLM as the central reasoning orchestrator.
2. Data Flows: How unstructured user inputs (text/voice) are transformed into structured API calls (stock, supplier, PO), with state persisted across the workflow.
3. Model Process: The end-to-end ReAct (Reason + Act) loop: listen → reason → orchestrate → observe → next step, including failure handling and human escalation.
4. Role of Reference: This document serves as a single source of truth for developers and testers to understand InvAgent's high-level architecture, data contracts, and AI integration patterns.

1.2. Background:

No previous software version exists. The baseline is the *manual workflow*: human checks stock → decides reorder → emails/calls supplier → waits for quote → gets approval → creates PO.

New Capabilities Introduced

- Unstructured (text/voice) intake → structured workflow.
- GLM-driven multi-step reasoning & dynamic API orchestration.
- Stateful session memory across workflow steps.
- Automatic failure recovery (retry, clarify, escalate).
- Structured PO generation.

© UMHackathon 2026

Email: umhackathon@um.edu.my

1.3. *Target Stakeholder*

Stakeholders	***Roles***	***Expectations***
Warehouse Supervisor	Triggers reorders via natural language ("low on rice")	System understands urgency, checks stock, and suggests reorder without manual data entry.
Procurement Officer	Approves POs, handles supplier failures	Only high-value orders require approval; system escalates failures with full context.
Small Retail Owner	No dedicated procurement staff; needs simple ops	"Just work" – type a message, get a PO. No training required.
Operations Manager	Oversees multiple workflows, monitors performance	Dashboard shows workflow state, token costs, API success rates, and audit logs.
Development Team	Builds and maintains InvAgent	Clear API contracts, modular prompt design, mockable GLM responses for testing, and documented state machine.
QA Team	Validates system behavior across all risks	Defined test scope (happy path, negative, NFR), AI output pass rate ≥80%, hallucination detection criteria.

© UMHackathon 2026

2. System Architecture & Design

2.1. High-Level Architecture

2.1.1. Overview:

Type	Details
System	Web-based dashboard + optional voice/text interface (no native mobile yet)
Architecture	Event-driven, LLM-centric orchestration with lightweight stateful backend

InvAgent is structured as a stateless API layer + stateful workflow engine where GLM (Z.AI) is the sole reasoning agent. The system does NOT use a generic "AI module"; instead, GLM is integrated as the orchestration service that decides which tool to call, when to ask for clarification, and when to escalate.

All clients (React dashboard, test harness, potential Slack/WhatsApp bot) communicate via a FastAPI Backend. The backend:

- Manages session state (Redis).
- Constructs prompts with ReAct pattern + compressed state JSON.
- Validates GLM outputs against a function-calling schema.
- Executes actual API calls (Stock DB, Supplier API, PO system, Approval Queue).
- Handles retries, fallbacks, and human escalation.

2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic “AI Module”. The following Dependency diagram includes:

2.1.3 Dependency Diagram

How prompts are constructed & sent to GLM API:

- System prompt: fixed (role, tools, ReAct instructions, few-shot examples).
- User prompt: compressed state JSON + new user input (text/voice transcript).
- Sent via HTTPS POST to GLM API with temperature=0.2, max_tokens=1000.

© UMHackathon 2026

What goes into the context window:

- System prompt (~500 tokens)
- Compressed workflow state JSON (e.g., `{step: "awaiting_approval", item: "rice", qty: 50}`) (~50 tokens)
- Last 2 user-assistant exchanges (~300 tokens)
- Current user input (~100 tokens)
- Tool output (e.g., `check_stock` result) (~100 tokens)
- Total target: < 1000 tokens, max 8K tokens.

How responses are parsed & passed:

- GLM returns JSON with `{"action": "tool_name", "parameters": {...}}` or `{"ask_user": "question"}`.
- Backend validates against Pydantic schema. If invalid → retry with lower temperature.
- If `action` is valid, backend calls the corresponding API.
- API result is fed back into the next GLM call.

Token limit enforcement & chunking:

- Pre-flight check: if input tokens >8000, reject with message: *"Input too large. Please summarize or upload as a file."*
- No automatic chunking for GLM (because inventory workflows need full context). Oversized email threads are summarized by a separate helper LLM call before injection.

API calls between components:
[Client] → [FastAPI] → [Redis (state)]
[FastAPI] → [GLM API]
[FastAPI] → [Stock API]
[FastAPI] → [Supplier API]
[FastAPI] → [Approval Queue (WhatsApp/Email)]
[FastAPI] → [PO System API]
[FastAPI] → [Human Escalation Channel]

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2.2. Sequence Diagram (Happy Path: Voice-to-Procurement)

2.2.2.1. User: *"We're low on 500ml bottles, maybe 20 boxes."*

Step	Actor	Action	Data Flow
1	User	Speaks into interface → STT converts to text	"We're low on 500ml bottles, maybe 20 boxes"
2	Backend	Loads state from Redis (idle). Constructs ReAct prompt.	{step: "idle", user_input: "..."}
3	Backend → GLM	POST /generate (prompt)	System prompt + state + user input
4	GLM → Backend	{"action": "check_stock", "parameters": {"item": "500ml bottle"}}	Valid JSON
5	Backend	Calls Stock API	GET /stock?item=500ml_bottle → {current: 10}

© UMHackathon 2026

Email: umhackathon@um.edu.my

6	Backend → GLM	Feeds tool result: {current: 10, requested: 20 boxes}	"You have 10 boxes. Reorder 20 boxes?"
7	GLM → Backend	{"action": "list_suppliers", "parameters": {"item": "500ml bottle"}}	
8	Backend	Calls Supplier API	Returns [{name: "LocalMart", price: 2.5}, ...]
9	Backend → GLM	Feeds supplier list	
10	GLM → Backend	{"action": "request_approval", "parameters": {"qty": 20, "total": 50}}	If total < RM10k → auto-approve, else wait.
11	Backend	Sends approval request to Procurement Manager via WhatsApp	"Approve PO for 20 boxes of 500ml bottles? Total RM50"
12	Manager	Approves	Webhook → Backend

© UMHackathon 2026

Email: umhackathon@um.edu.my

13	Backend → GLM	Approval confirmed	
14	GLM → Backend	{"action": "create_po", "parameters": {...}}	
15	Backend	Calls PO System API → PO created	Returns PO number
16	Backend → User	Dashboard notification + confirmation message	"PO #INV-001 created for 20 boxes."

2.2.4 Technological Stack

Layer	Technology	Justification
Frontend	React + TypeScript (Web Dashboard)	Rapid prototyping, state management, and dashboard visualizations for workflow monitoring.
Backend	FastAPI (Python)	Async support for GLM API calls, easy function-calling schema

© UMHackathon 2026

		validation, and Webhook handling.
State Store	Redis (in-memory with optional persistence)	Low-latency session state (ttl=1 hour), compressed JSON storage, supports server restart recovery.
Database (log)	PostgreSQL (for audit logs only, not workflow state)	Stores completed workflow logs, token usage, and API call traces for QA & monitoring.
GLM API	Z.AI GLM (function-calling enabled)	Sole reasoning engine. Justified in PRD.
External APIs	Stock API (mock), Supplier API (mock), PO System API (mock), WhatsApp Business API	All mocked in prototype but designed for real integration.
Deployment	Docker Compose (local) + GitHub Actions CI. AWS target (preliminary simulation).	Matches QAT's CI/CD gates.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2.5 Data Flow Diagram (DFD) – Level 0 (Context)

```text
[User] → (unstructured text/voice) → [InvAgent System]
[InvAgent System] → (PO, approval request) → [User]
[InvAgent System] → (stock query) ↔ [Stock DB]
[InvAgent System] → (supplier query) ↔ [Supplier DB]
[InvAgent System] → (create PO) ↔ [Accounting System]

[InvAgent System] → (log event) → [Audit Log]
```

2.2.6 Normalized Database Schema (Audit & Logging only – 3NF)

Note: Workflow state is stored in Redis (key-value), not relational DB, to meet latency requirements. PostgreSQL is used for audit & analytics.

Table: `sessions`

- `session_id` (PK)
- `user_id`
- `start_time`
- `end_time`
- `total_tokens_in`
- `total_tokens_out`
- `status` (completed, failed, escalated)

Table: `workflow_steps`

- `step_id` (PK)
- `session_id` (FK → sessions)
- `step_name` (check_stock, list_suppliers, request_approval, create_po)
- `glm_action` (JSON)
- `api_call` (endpoint)
- `api_response` (JSON)
- `timestamp`
- `success` (boolean)
- `retry_count`

© UMHackathon 2026

Table: `suppliers` (reference)

- `supplier_id` (PK)
- `name`
- `api_endpoint`
- `max_capacity_kg`

Table: `inventory_items` (reference)

- `item_id` (PK)
- `name`
- `threshold_quantity`
- `unit`

No direct M: N relationships needed in audit schema. This satisfies 3NF.

3. *Functional Requirements & Scope*

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

#	***Features***	***Description***
1	Unstructured Intake	Parse text/voice (simulated) inputs: extract item, quantity, urgency.
2	Multi-step GLM Orchestration	GLM decides: check_stock → list_suppliers → request_approval → create_po (ReAct pattern).
3	Stateful Workflow	Persist state in Redis across turns; survive server restart.
4	Failure Handling	Missing data → GLM asks for clarification. API failure → retry (3x) → escalate to a human.
5	Structured PO Creation	Generate PO JSON → call mock accounting API → return PO number.

© UMHackathon 2026

3.2. *Non-Functional Requirements (NFRs)*

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Reliability	99% of happy-path workflows complete without human intervention for orders < RM10k.	Retry logic (exponential backoff), state persistence, and human escalation on failure.
Performance	Each GLM reasoning step < 5 sec (p95). API calls (stock, supplier) < 200ms.	Async FastAPI, Redis state, connection pooling for APIs, and caching frequent stock checks (5min TTL).
Cost Efficiency	Average token cost < $0.02 per session (2K input + 1K output).	Compressed state JSON (instead of full chat history), caching, prompt truncation, and token counting before GLM call. Reject >8K input.
Security	No PII (supplier names hashed) in GLM prompts. API keys are never logged.	Prompt sanitization middleware, environment variables for keys, and audit logs exclude raw inputs.
Hallucination	Hallucination rate < 5% of total responses.	Function-calling schema validation, low temperature (0.2), few-shot examples, output retry on schema failure, confidence scoring (where available).
Maintainability	Each tool (check_stock, list_suppliers, etc.) is implementable as a separate pluggable function.	Tool registry pattern. New tools added without changing GLM prompt logic (via tool schema injection). CI/CD runs unit tests on each tool in isolation.

© UMHackathon 2026

3.3. *Out of Scope / Future Enhancements*

a. Multi-warehouse conflict resolution.
b. Predictive demand forecasting.
c. Direct payment processing.
d. Physical IoT/sensor integration.
e. Native Slack/WhatsApp bot (will use webhook stubs in MVP).
f. Real voice STT (simulated with text input in MVP).

4. *Monitor, Evaluation, Assumptions & Dependencies*

4.1. *Technical Evaluation*

Strategy	Implementation
Grayscale Rollout	New prompt versions are released to 10% of sessions first; monitor the hallucination rate and cost.
A/B Testing	Compare two prompt templates (e.g., different few-shot examples) for extraction accuracy.
Emergency Rollback (ER)	If GLM hallucination rate > 10% or API error rate > 20% in 5 min → auto-rollback to last known good release (Golden Release).
Priority Matrix (P0)	Trigger: GLM returns an invalid function call schema for 3 consecutive requests. Action: Block pipeline, alert on-call.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Priority Matrix (P1)	Trigger: Stock API timeout > 2 sec for 10% of calls. Action: Retry, then degrade to manual stock input prompt.

4.2. Monitoring

4.2.1.

Metric	Tool/Method	Threshold & Action
GLM response latency	FastAPI middleware + logs	p95 > 5 sec → alert. >7 sec → circuit breaker (fallback to human).
Hallucination rate	Schema validation failures/total	>5% → trigger A/B test of prompt. >10% → emergency rollback.
Token cost per session	Count input/output tokens via GLM API	>$0.05 → alert dev team. >$0.10 → reject new sessions until investigated.
API failure cascade	Retry counter + log	Any API fails 3x in one workflow → alert + escalate to human (via webhook).

© UMHackathon 2026

Email: umhackathon@um.edu.my

State loss	Redis health check	Redis unavailable → reject new workflows, but complete existing in-memory ones.

4.3. Assumptions

a. Stable internet connectivity between backend, GLM API, and external APIs.
b. Supplier and Stock APIs have predictable response times (<500ms).
c. Human approvers respond to WhatsApp/email approval requests within 10 minutes (default timeout).
d. GLM's function-calling outputs follow the requested JSON schema reliably when the temperature is low.
e. User inputs are in English or Malay (basic); complex code-switching may be rejected.

© UMHackathon 2026

4.4. *External Dependencies:*

Tools	*Purpose*	*Risks*
Z Ai GLM API	Core reasoning, function calling, intent extraction.	High. API rate limit → queue requests. API downtime → graceful degradation: "AI offline. Please use manual PO form."
Google Maps API	*Tracking rider location and optimize the route*	*Medium level- if real time tracking is being disabled.*

5. *Project Management & Team Contributions*

5.1. *Project Timeline:*

Day(s)	Phase	Key Activities
1-2	Planning & Setup	Define state machine, tool schemas, GLM prompt templates, CI pipeline (GitHub Actions).
3-4	Core Backend + GLM Integration	FastAPI endpoints, Redis state, GLM calling loop, schema validation.
5-6	Tool Implementation	`check_stock`, `list_suppliers`, `request_approval`, `create_po` (mock APIs).

© UMHackathon 2026

7	Failure Handling & Escalation	Retry logic, human escalation path, missing data clarification.
8	Frontend Dashboard (basic)	Display workflow state, PO confirmation, simple input box.
9	QA & Testing (per QAT)	Run all 15+ test cases: happy path, negative, NFR, AI boundary tests. Meet CI/CD gates.
10	Demo Prep & Deployment	Documentation finalization, video recording, live demo (Docker Compose + mock APIs).

5.2. Team Members' Role:

Aleeya Nazirah binti Jamil - Project Manager
Ahmad Naufal bin Azlan - Lead Developer
Humaira Adani binti Muhamad Azri - Assistant Developer
Raihanah binti Rozaini - Pitch and Documentation Lead
Raja Athira Sofea binti Raja Azlan - Database and Presentation Lead

5.3. Recommendations:

6. Prompt Caching: Cache GLM responses for identical `(state, user_input)` pairs for 5 minutes (reduces cost & latency for repeated stock checks).
7. Asynchronous Approval: Use webhooks + polling for approval status instead of blocking the GLM loop.
8. Token Budgeting: Implement a middleware that rejects any user input that would cause total prompt >6000 tokens, with a clear error message (instead of failing silently).
9. Horizontal Scaling: Design state store (Redis) as external; backend instances stateless. Use a message queue (RabbitMQ) for GLM requests if rate limits become an issue.

© UMHackathon 2026